

Mathematics: analysis and approaches HL | Exploration

Is gradient descent or simulated annealing a better optimization algorithm for machine learning-based regression analysis in terms of accuracy and efficiency?

Candidate number: jqx050

Session: May 2024

Page count: 16

Contents

1 Introduction	1
2 Methodology	1
2.1 Regression and optimization	1
2.2 Loss functions	2
2.3 Optimization algorithms	2
2.4 Modeling the problem	3
3 Gradient descent (GD)	4
3.1 The gradient descent algorithm	5
3.2 Gradient descent on our problem	9
4 Simulated annealing (SA)	10
4.1 The simulated annealing algorithm	10
4.2 Simulated annealing on our problem	13
5 Analysis	14
6 Conclusion	16
Bibliography	17
Appendix I: Excerpt of dataset	18
Appendix II: Algorithm implementations and visualization code	19

1 | Introduction

Machine learning (ML) is a branch of artificial intelligence that uses statistical techniques to give computer systems the ability to “learn” from and make predictions on data, without being explicitly programmed. Today, machine learning models are used in our everyday lives, from personalized recommendations on Netflix to facial recognition on our phones.

As someone studying machine learning, the very first ML task I learned to solve was **regression analysis**, which is the statistical process of finding a correlation between a dependent variable and one or more independent variables (Freedman, 2009). This is often used to predict future values of the dependent variable based on values of the independent variable(s), for instance predicting the price of a house based on its size and location. The process of training a regression model involves **optimization**. This optimization can be done using a variety of algorithms, but the end goal is always the same: to find the best parameters of the model by minimizing the difference between the model’s predictions and the actual values of the dependent variable.

In that study, I have come to value understanding the underlying algorithms, because knowing the limitations and strengths of different algorithms can help data scientists choose the right one for a given problem. For this reason, this exploration will compare two common **optimization algorithms**, gradient descent and simulated annealing, in terms of their **accuracy** and **efficiency** when applied to machine learning-based regression analysis. In the real world, this translates to how accurate a model’s predictions are on new data, and how quickly it can be “trained” to do so.

2 | Methodology

2.1 | Regression and optimization

In applied statistics (which blends mathematics and computer science), **regression** is a **supervised learning**¹ algorithm that models the relationship between a dependent variable y and an independent variable(s) x . The model can be represented visually as a line or curve that best fits the data.

To compute this regression, we optimize. **Optimization** is the process of finding the best solution to a problem, which for our problem is finding the equation of the line that best fits the data, according to some measure. In machine learning, this measure is called a loss function; and so the goal of optimization becomes minimizing the **loss function**.

¹**Supervised learning** is a category of machine learning tasks where a model learns a function based on example input-output pairs (from the **training data**).

2.2 | Loss functions

There are many types of loss functions, which depend on the problem and type of data. This includes “square of Euclidean distance, cross-entropy, contrast loss, hinge loss, information gain,” etc (Sun et al., 2019).

A common loss function is the sum of squared residuals, defined as

$$L = \sum_{i=1}^n (y_i - f(x_i))^2 \quad (1)$$

where y_i is the i^{th} actual value of the variable we are trying to predict (“ground truth”), $f(x_i)$ is the predicted value of y_i given the i^{th} value of the independent variable x , and n is the number of data points.

This loss function was also chosen because it is differentiable, which is a requirement for gradient-based methods like gradient descent, which we are comparing.

2.3 | Optimization algorithms

Optimization algorithms are used to minimize the loss function and in turn, solve the optimization problem. While they can work on varying principles, they can broadly be categorized into **deterministic** and **stochastic** algorithms. Deterministic algorithms are those that follow a fixed set of rules to find the minimum of the loss function, while stochastic algorithms use randomness to find the minimum (Sun et al., 2019).

To look at the effect of each optimization algorithm on the performance of a regression model, we will apply both a deterministic and a stochastic algorithm to a real-world dataset of information on cars. We will then compare the results of the two algorithms to an analytical solution (see Section 5) to determine which algorithm is more accurate and efficient.

An **accurate** regression model for the problem would help us accurately predict the fuel efficiency of a car given its horsepower, both interpolating and extrapolating. This can be measured statistically, with the **coefficient of determination** R^2 , as well as in comparison to the analytical solution.

An **efficient** regression model for the problem would be able to find the best-fit line with as few iterations as possible. This can be measured statistically by the number of iterations taken to converge on a solution, as well as the rate of convergence.

2.4 | Modeling the problem

“Car information dataset” is a dataset published on Kaggle, a data aggregator, by Tawfik Elmetwally in 2023 (Elmetwally, 2023). The dataset contains information on 399 cars, including their make, model, engine displacement and cylinder count, horsepower, fuel efficiency, and more. Take two variables, horsepower and fuel efficiency. Say we want to model the relationship between these two variables, and so let us define our problem to be modeling this relationship between the horsepower rating (in hp) of a car and its fuel efficiency (in mpg).

Extracting the relevant columns (mpg and horsepower) from the dataset and plotting them on a set of axes, we get the following scatterplot:

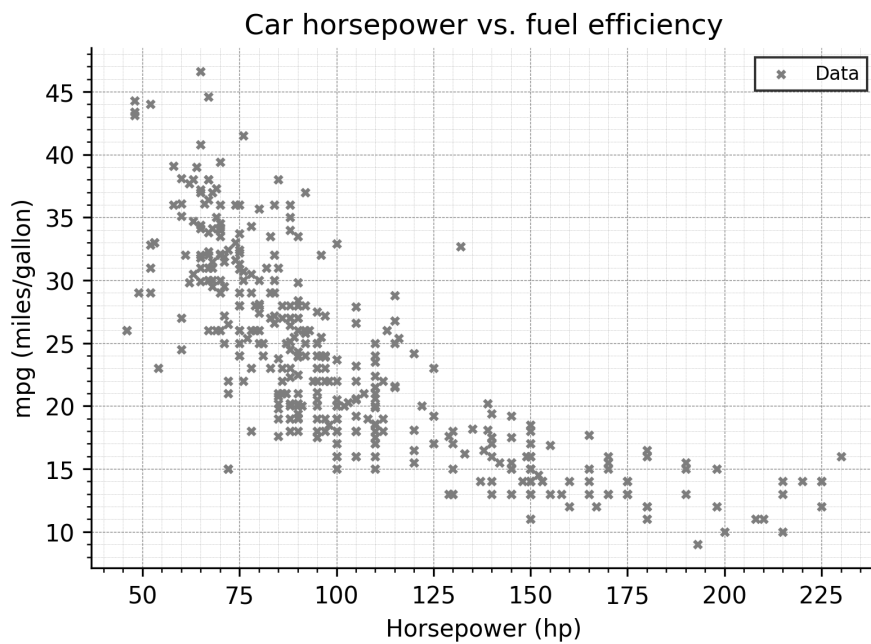


Figure 1: Scatterplot of car data. Derived from dataset published by Elmetwally (2023).

In Figure 1, we observe a negative exponential relationship between the two variables, where as horsepower increases, fuel efficiency decreases at a slowing rate. Hence, we will attempt to optimize an exponential regression model to fit the data.

This will be in the form of the equation

$$f(x) = a \cdot e^{b(x-h)} + k.$$

Parameter	Domain	Range
$f(x)$	$x \in \mathbb{R}$	$f(x) > k$

Parameter	Meaning	Domain
a	Scaling factor	$a \in \mathbb{R}$
b	Growth factor	$b \in \mathbb{R}$
h	Horizontal shift	$h \in \mathbb{R}$
k	Vertical shift	$k \in \mathbb{R}$

Now, we define the loss function for our specific problem. In this case, y is the fuel efficiency of the car, x is the horsepower rating of the car, and $f(x)$ is the predicted fuel efficiency of the car given its horsepower rating. Each squared residual comes from the difference between the actual value of y and the predicted value of y , $f(x_i)$.

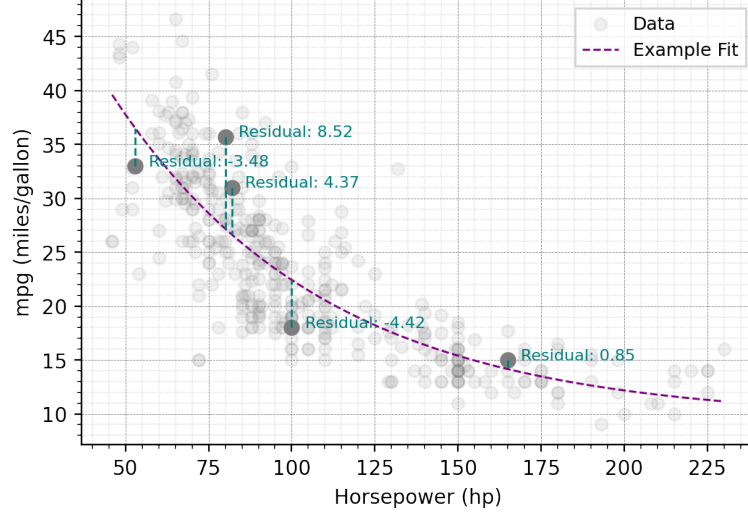


Figure 2: As an example, residuals between datapoints and an example regression line

Therefore, the generic loss function (Equation 1) can be rewritten as

$$L = \sum_{i=1}^n (y_i - a \cdot e^{b(x-h)} - k)^2 \quad (2)$$

When we optimize this loss function, we can calculate the summation, substituting y_i and x_i with each datapoint's value, which leaves the loss function in terms of the parameters a , b , h , k as the targets for optimization.

3 | Gradient descent (GD)

Gradient descent is an optimization algorithm that minimizes a function by taking steps proportional to the negative of the gradient of the function at the current point.

For convex loss functions with a clear global minimum, gradient descent should be powerful and efficient, because it will deterministically find the minimum of the loss function. However, in non-convex loss functions or ones that have multiple local minima, it can be slow to converge or get “stuck” in local minima and never converge on the global minimum (Sun et al., 2019). We will soon infer whether this is the case for our problem.

3.1 | The gradient descent algorithm

Initialization

We first define an initial solution (a mathematical “**guess**” based on intuition and visual data), where we choose arbitrary but sensible values for the parameters. Let’s call these a_0 , b_0 , h_0 , and k_0 . In choosing guesses, guesses closer to the actual values will allow the optimization algorithm to converge on a solution in fewer iterations, so the same guess will be used for GD and SA.

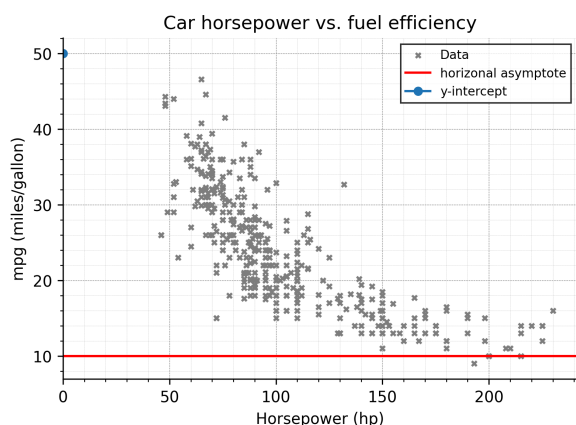


Figure 3: Visual estimation of parameters

→ Let’s choose:

- $a_0 = 1$
- $b_0 = -0.05$, as we recognize an exponential decay shape.
- $h_0 = 80$, as that brings the y-intercept around 50 mpg.
- $k_0 = 10$, as the horizontal asymptote seems to exist around 10 mpg.

These parameter values will be continually adjusted in each iteration of the algorithms, and the final values after the loss function converges can be substituted into the original identity function $f(x)$ to get the equation of the regression.

Computing the gradient

Taking the partial derivative of the loss function with respect to each parameter gives us the instantaneous rate of change of each parameter, which is the slope of the loss function at that point. Partial derivatives are used because the loss function is in terms of 4 parameters, and we want to find the gradient of each while the others are held constant.

Together, the partial derivatives make up the **gradient** of the loss function, which “denotes the direction of greatest change of a scalar function” (Bachman, 2007) — essentially a vector that points in the direction of the steepest ascent of the loss function. The idea with GD is that, by taking steps in the opposite direction of the gradient, we will eventually reach the minimum of the loss function, and in the process obtain the parameters a , b , h , and k that provide that minimum. This is our optimal solution.

Following this, we compute the derivatives.

→ **Derivative with respect to a**

From Equation 2, we have

$$L(a, b, h, k) = \sum (y_i - a \cdot e^{b(x_i-h)} - k)^2$$

Differentiating L with respect to a , we can apply the chain rule.

$$\begin{aligned} \frac{\partial L}{\partial a} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -e^{b(x_i-h)} \\ \frac{\partial L}{\partial a} &= -2 \times \sum e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (3)$$

Following the same mathematical approach, the derivatives with respect to b , h , and k can be computed:

→ **Derivative with respect to b**

$$\begin{aligned} \frac{\partial L}{\partial b} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times a \cdot e^{b(x_i-h)} \cdot (x_i - h) \\ \frac{\partial L}{\partial b} &= -2 \times \sum a \cdot e^{b(x_i-h)} (x_i - h) (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (4)$$

→ **Derivative with respect to h**

$$\begin{aligned} \frac{\partial L}{\partial h} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -ab \cdot e^{b(x_i-h)} \\ \frac{\partial L}{\partial h} &= 2 \times \sum ab \cdot e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (5)$$

→ **Derivative with respect to k**

$$\begin{aligned} \frac{\partial L}{\partial k} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -1 \\ \frac{\partial L}{\partial k} &= -2 \times \sum (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (6)$$

Altogether, the gradient is

$$\begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial h} \\ \frac{\partial L}{\partial k} \end{bmatrix} = \begin{bmatrix} -2 \times \sum e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \\ -2 \times \sum a \cdot e^{b(x_i-h)} (x_i - h) (y_i - a \cdot e^{b(x_i-h)} - k) \\ 2 \times \sum ab \cdot e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \\ -2 \times \sum (y_i - a \cdot e^{b(x_i-h)} - k) \end{bmatrix} \quad (7)$$

The current parameters are then substituted into the gradient.

→ For instance, substituting the initial parameters a_0 , b_0 , h_0 , and k_0 into the gradient and computing the summation over every data point, we get the following value for the gradient:

$$\begin{bmatrix} 15921.6831 \\ 87049814.8282 \\ 796.0842 \\ 17774.9488 \end{bmatrix} \quad (8)$$

Parameter update

The gradient is not used to adjust the parameters directly, however. Rather, the gradient is multiplied by a scalar, the **learning rate** (γ), which controls the rate of parameter change. A balance must be found with the learning rate, as a learning rate too small “will result in a slower convergence rate”, while a learning rate too large can cause overshooting, hindering convergence (Sun et al., 2019). The product of the gradient and learning rate is the **step size**, which is the actual amount each parameter is changed per iteration.

$$\text{Step size} = \gamma \times \begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial h} \\ \frac{\partial L}{\partial k} \end{bmatrix} \quad (9)$$

The step sizes are then subtracted from the original parameters (the parameters from the previous iteration or the initial values) to get new values. If n is the current iteration, this can be defined as

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} a_{n-1} \\ b_{n-1} \\ h_{n-1} \\ k_{n-1} \end{bmatrix} - \gamma \times \begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial h} \\ \frac{\partial L}{\partial k} \end{bmatrix} \quad (10)$$

→ Continuing the example iteration, let's set the learning rate γ to 10^{-9} initially (from Sun et al. (2019)). Following the steps outlined above, we calculate the step size and then the new values for each parameter.

$$\text{Step size} = 10^{-9} \times \begin{bmatrix} 15921.6831 \\ 87049814.8282 \\ 796.0842 \\ 17774.9488 \end{bmatrix} \quad (11)$$

and the new parameter values (since we don't have a previous iteration, we use the initial values for iteration $n - 1$):

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} 1 \\ -0.05 \\ 80 \\ 10 \end{bmatrix} - 10^{-9} \times \begin{bmatrix} 15921.6831 \\ 87049814.8282 \\ 796.0842 \\ 17774.9488 \end{bmatrix} = \begin{bmatrix} 0.999984078317 \\ -0.13704981428 \\ 79.9999992039 \\ 9.99998222505 \end{bmatrix} \quad (12)$$

After one iteration of GD, we now have a refinement of our regression equation:

$$f(x) = 0.999984078317 \times e^{-0.13704981428(x-79.9999992039)} + 9.99998222505 \quad (13)$$

Over this single iteration, the changes are minuscule, but over many iterations, the loss function eventually converges to a minimum.

Convergence check and iteration

This process of calculating the gradient, step size, and finally the new parameter values, is repeated until the loss function converges to a minimum, where the loss function approaches a value that is either the global minimum or a local minimum, at which further adjustments to the model parameters result in negligible or no improvement in the loss function.

Convergence can be determined by setting a threshold for the step size (when it is sufficiently small) or by limiting the number of iterations.

3.2 | Gradient descent on our problem

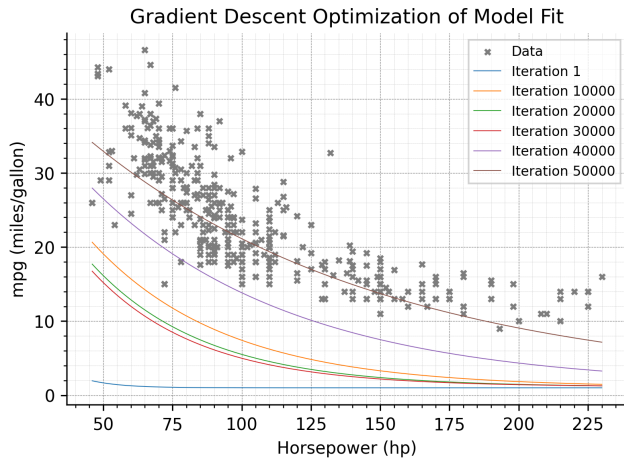


Figure 4: Gradient descent on our problem

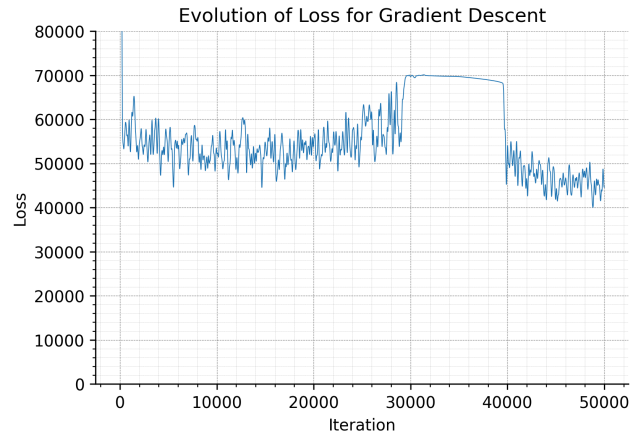


Figure 5: Loss function over iterations

Programmatically applying the gradient descent process on the data, the algorithm ran through 50,000 iterations, which is visualized above. Figure 4 shows the progression of the regression line over the iterations, while Figure 5 shows the loss function over the iterations. Recall that the loss function is the sum of squared residuals, and the goal of the algorithm was to minimize this function.

While the difference between each iteration is minimal, adjustments in the parameters by the gradient descent algorithm lead to the parameters slowly converging over many iterations on a final solution on the last iteration (brown line, Iteration 50000).

This convergence is reflected in the loss graph, which decreases over time although with many fluctuations. It also does not appear to converge, with no clear minimum or horizontal asymptote visible. Between the ~30,000th to ~40,000th iteration, the loss function was flat, potentially indicating that the algorithm was stuck in a local minimum.

Statistically, the solution from Iteration 50,000 yielded an R^2 value (coefficient of determination) of 0.603810, meaning about 60.38% of the variance in the dependent variable (mpg) is explained by the regression model; the model captured a significant portion (but not all) of variability in the data. This represents an acceptable fit, but it remains to be seen how this compares to the results of simulated annealing and how close both algorithms are to the analytical solution.

4 | Simulated annealing (SA)

In contrast to gradient descent, which is a deterministic gradient-based method, simulated annealing is a probabilistic technique inspired by metallurgy that makes random changes to the solution and uses the concept of ‘temperature’ to control the search process. This should represent a radically different optimization approach: the temperature makes the algorithm more likely to accept “worse” (between iterations) solutions at the beginning of the search process, and more conservative later on when a better approximation to the global minimum of the loss function is found.

4.1 | The simulated annealing algorithm

Initialization

Like in GD, we start with an initial solution, the guess. Let’s use the same guess as before so that the algorithms can be directly compared.

We also set an initially high **temperature** T_0 . The idea is that, at high temperatures, the algorithm is more likely to accept changes that increase the function value (or ‘energy’), allowing it to explore a wide range of solutions. As the temperature decreases, the algorithm becomes more selective, focusing on refining and improving the current solution. This process helps SA to potentially escape local minima and converge towards a global minimum. This balance of exploration (searching new areas of the solution space) and exploitation (refining solutions in promising areas) is useful, especially when the function’s landscape is complex with many local minima (Kirkpatrick et al., 1983).

→ Let’s set the initial temperature T_0 to 500, from Kirkpatrick et al. (1983).

Iteration

In each iteration, we make a random perturbation to the current solution in the form of adding a random value to each parameter. This removes any inherent bias in the direction of the search, allowing for an unbiased exploration of the solution space.

In my implementation, I will generate random numbers from a normal (Gaussian) distribution with $\mu = 0$ and $\sigma = 1$, or $N \sim (0, 1)$, because it is continuous, and adjustments in both the positive and negative directions have equal probability.

Hence, if n is the current iteration, the perturbed values are

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} a_{n-1} + \Delta a \\ b_{n-1} + \Delta b \\ h_{n-1} + \Delta h \\ k_{n-1} + \Delta k \end{bmatrix} \quad (14)$$

where $\Delta a \sim N(0, 1)$ and so forth.

→ Generating random perturbations in the program implementation, we get the following perturbed values.

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} 1 + \Delta a \\ -0.05 + \Delta b \\ 80 + \Delta h \\ 10 + \Delta k \end{bmatrix} = \begin{bmatrix} 1.54626094 \\ -0.08207048 \\ 80.84881705 \\ 10.03802701 \end{bmatrix} \quad (15)$$

Evolution and acceptance

After perturbation, the new candidate solution is evaluated by the difference in the loss function, ΔL . If n is the current iteration, $\Delta L_n = L_{n-1} - L_n$.

If the candidate solution is better than the existing solution, that is, $\Delta L > 0$, then the candidate solution is accepted as the new current solution.

If the candidate solution is worse than the existing solution, that is, $\Delta L < 0$, then the candidate solution is accepted as the new current solution with a probability of

$$P(\text{accept}) = e^{-\frac{\Delta L}{T}} \quad (16)$$

where T is the current temperature. Recall that this allows SA to escape local minima, as in practice, it is more likely to accept worse solutions early on, and then become more “selective” as the temperature decreases. As the temperature cools, the probability of accepting worse solutions decreases, and the algorithm becomes more conservative, focusing on refining and improving the current solution.

→ Subtracting the new loss function value from the loss function value obtained from the initial guess, we get $\Delta L = 57943.6433449937$.

Because $\Delta L > 0$ in this iteration, the candidate solution is better than the current solution, so we accept the candidate solution as the new current solution.

Cooling

At the end of each iteration, the temperature is decreased by a cooling factor α . This is done to reduce the probability of accepting worse solutions as the algorithm progresses.

For instance, a simple exponential decay might be

$$T_{\text{new}} = \alpha \times T_{\text{old}} \quad (17)$$

where α is a constant < 1 , controlling the rate of temperature decrease.

→ Let's choose $\alpha = 0.98$, to give the algorithm more time to explore other minima.

The new temperature is then

$$\begin{aligned} T_{\text{new}} &= 0.98 \times T_{\text{old}} \\ &= 0.98 \times 500 \\ &= 490 \end{aligned} \quad (18)$$

This completes one iteration, which leaves us with a refinement of our regression equation:

$$f(x) = 1.54626094 \times e^{-0.08207048(x-80.84881705)} + 10.03802701 \quad (19)$$

Termination

The algorithm repeats the iteration step until a stopping criterion is met. This could be a sufficiently low temperature, a maximum number of iterations, or signs of convergence (e.g. minimal change in the loss function over a certain number of iterations).

→ Stopping criteria are usually chosen through trial and error based on the problem and the desired trade-off between accuracy and efficiency, but for the purpose of this exploration, we will use a maximum number of iterations as the stopping criteria to directly compare the performance of the two algorithms.

Thus, we will run the simulated annealing for the same number of iterations as gradient descent, 50,000.

4.2 | Simulated annealing on our problem

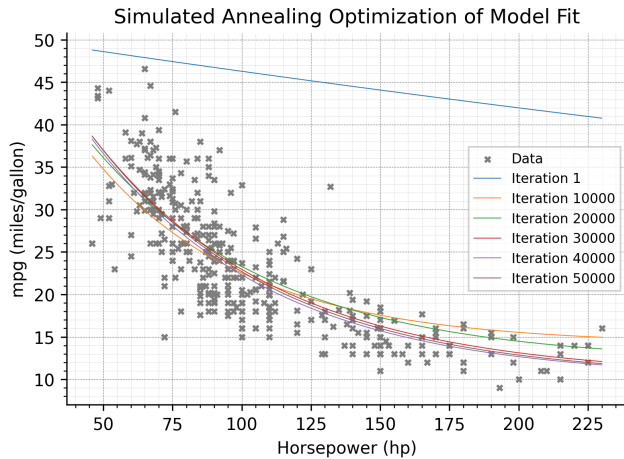


Figure 6: Simulated annealing on our problem

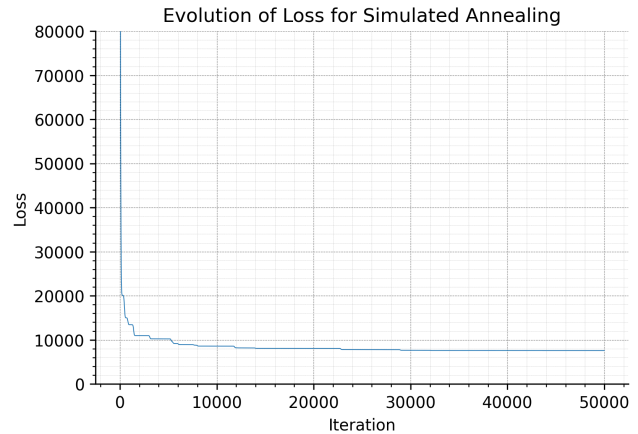


Figure 7: Loss function over iterations

Similarly to the GD algorithm, simulated annealing was programmatically run for 50,000 iterations. Figure 6 shows the progression of the regression line over the iterations, while Figure 7 shows the loss function over the iterations.

As seen in Figure 6, the regression line of iterations 10,000 and above appear to fit the scatterplot well. The solution from Iteration 50,000 yielded an R^2 value (coefficient of determination) of 0.661143, statistically indicating approximately 66.11% of the variance in the dependent variable is explained by the regression model. This is higher than the R^2 value from the GD algorithm, which suggests that the SA algorithm was more accurate.

SA was also able to find this solution faster, as the loss function converged to a minimum by the 10,000th iteration. This is reflected in the loss graph, which showed a steep drop, indicating that SA was quickly able to find a satisfactory solution within 10,000 iterations before converging on a minimum, which appears as a horizontal asymptote around $L = 8000$.

5 | Analysis

We have now seen the results of both gradient descent (GD) and simulated annealing (SA). To evaluate their performance and answer the research question “Is gradient descent or simulated annealing a better optimization algorithm for machine learning-based regression analysis in terms of accuracy and efficiency?”, we will now compare their results to an analytical solution calculated using the Levenberg-Marquardt algorithm, a common method for finding high precision approximations of the exact solution (Sun et al., 2019). This comparison is shown in Figure 8.

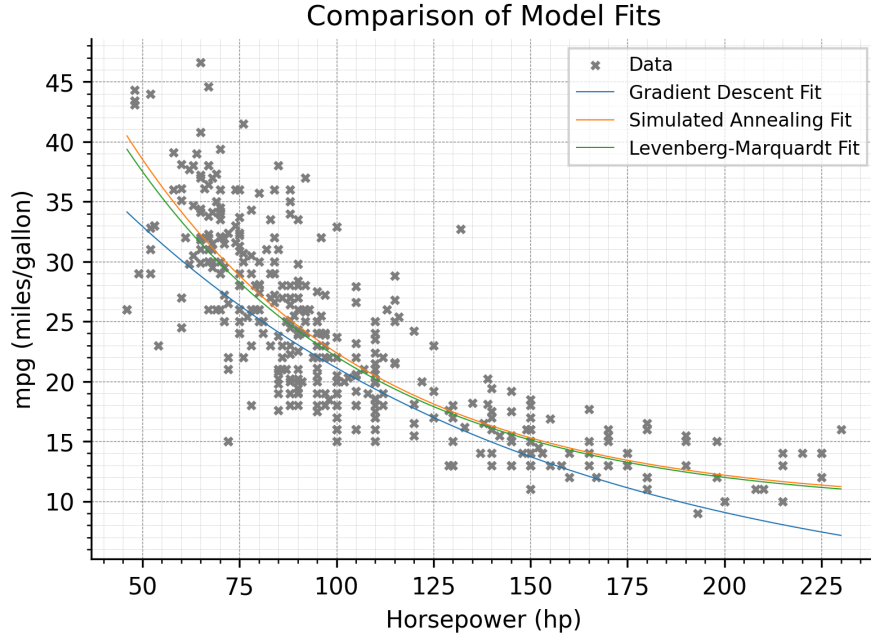


Figure 8: Comparison between two optimizers and the analytical line of best fit

Numerically, the final iteration of each algorithm resulted in the following parameter values, which directly translate to the regression functions seen plotted in Figure 8.

Algorithm	a	b	h	k	L (loss)	R^2
Gradient descent	50.010563	-0.009311	1.038900	1.226458	70446.186542	0.603810
Simulated annealing	54.825537	-0.015162	2.979578	10.094089	7579.026930	0.661143
Analytical solution (Levenberg-Marquardt)	41.927	-0.016	24.890	9.453	7514.335123	0.682711

Table 1: Final values for the parameters of the equation, the loss function L , and the coefficient of determination R^2

Comparing the accuracy of the solutions found by the two optimization algorithms to the analytical solution suggests that in this problem, simulated annealing was more accurate than gradient descent.

This is reflected in each solution's coefficient of determination. Simulated annealing was able to find a solution that was closer to the analytical solution, with an R^2 value of 0.661143 compared to gradient descent's lower R^2 value of 0.603810. Compared to the analytical solution, which had an R^2 value of 0.682711, this is a percentage discrepancy of 0.0316% for simulated annealing and 0.1156% for gradient descent.

This is also evidenced by the difference in the loss value between simulated annealing and the analytical solution; simulated annealing was able to find a solution that was closer to the analytical solution (the theoretical best solution), with a difference in the loss value to the analytical solution of 64.691807 compared to gradient descent's significantly higher difference of 62,931.851419.



Figure 9: Comparison of loss function over iterations

In comparing the efficiency of the two algorithms, simulated annealing was also more efficient than gradient descent. This is evidenced by the number of iterations taken to converge on a solution; simulated annealing was able to converge on a solution by the 10,000th iteration, while gradient descent took 50,000 iterations to converge on a solution that still led to noise in the loss function (it did not converge on a clear minimum). This is shown in the loss graph in Figure 9.

6 | Conclusion

The research question, “Is gradient descent or simulated annealing a better optimization algorithm for machine learning-based regression analysis in terms of accuracy and efficiency?” was answered by comparing the performance of the two algorithms to an analytical solution in Section 5. The analysis suggested that simulated annealing was both more accurate and efficient than gradient descent, for this problem. It was able to find a solution that was closer to the analytical solution, and in fewer iterations than gradient descent.

This may be explained by the fact that gradient descent is a deterministic algorithm that is limited in its ability to escape local minima, especially in cases of non-convex loss functions or when the loss function has multiple local minima. Simulated annealing, on the other hand, is a probabilistic algorithm that is better at escaping local minima early on, and this is reflected in the fact that it was able to find a solution that was closer to the analytical solution, and more efficiently, than gradient descent.

Understanding that simulated annealing may work better for non-convex loss functions or when the loss function has multiple local minima is an insight I find valuable for my future optimization projects. From these results, inferences can be made about which algorithm may be suitable for which problems, and this can be used to guide the choice of optimization algorithm in future analyses. At the same time, the results of the exploration may not generalize to all problems. There is a high degree of data and parameter specificity across optimization problems, and the performance of each algorithm will depend on the specific problem and the specific data.

For a larger scope investigation, I would have liked to explore other methods like genetic algorithms or particle swarm optimization to compare their performance to gradient descent and simulated annealing. This would have provided a more comprehensive understanding of the performance of different optimization algorithms in machine learning-based regression analysis. Also, I would have liked to explore the impact of the loss function itself on the performance of the optimization algorithms, and how the choice of loss function might impact the performance of the algorithms.

Extending the exploration even more, advanced optimization techniques combine multiple fundamental algorithms. For instance, the Levenberg-Marquardt algorithm we used as an analytical solution combines gradient descent and Newton’s method, which uses second-order derivatives. This strongly suggests that a combination of methods may be more effective than using a single method, and is worthwhile to explore in the future.

Bibliography

Bachman, D. (2007). *Advanced Calculus Demystified*. McGraw-Hill.

Elmetwally, T. (2023). *Car Information Dataset* [Computer software]. <https://www.kaggle.com/datasets/tawfikelmetwally/automobile-dataset>

Freedman, D. A. (2009). *Statistical Models: Theory and Practice*. Cambridge University Press.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>

Sun, S., Cao, Z., Zhu, H., & Zhao, J. (2019, October 23). *A Survey of Optimization Methods from a Machine Learning Perspective*. <http://arxiv.org/abs/1906.06821>

Appendix I: Excerpt of dataset

As an example, the following is an excerpt of the dataset. The full dataset contains 399 rows and 9 columns, and is available on Kaggle (Elmetwally, 2023).

```
name,mpg,cylinders,displacement,horsepower,weight,acceleration,model_year,origin
chevrolet chevelle malibu,18,8,307,130,3504,12,70,usa
buick skylark 320,15,8,350,165,3693,11.5,70,usa
plymouth satellite,18,8,318,150,3436,11,70,usa
amc rebel sst,16,8,304,150,3433,12,70,usa
ford torino,17,8,302,140,3449,10.5,70,usa
ford galaxie 500,15,8,429,198,4341,10,70,usa
chevrolet impala,14,8,454,220,4354,9,70,usa
plymouth fury iii,14,8,440,215,4312,8.5,70,usa
pontiac catalina,14,8,455,225,4425,10,70,usa
amc ambassador dpl,15,8,390,190,3850,8.5,70,usa
dodge challenger se,15,8,383,170,3563,10,70,usa
plymouth 'cuda 340,14,8,340,160,3609,8,70,usa
chevrolet monte carlo,15,8,400,150,3761,9.5,70,usa
buick estate wagon (sw),14,8,455,225,3086,10,70,usa
toyota corona mark ii,24,4,113,95,2372,15,70,japan
plymouth duster,22,6,198,95,2833,15.5,70,usa
amc hornet,18,6,199,97,2774,15.5,70,usa
ford maverick,21,6,200,85,2587,16,70,usa
datsun pl510,27,4,97,88,2130,14.5,70,japan
volkswagen 1131 deluxe sedan,26,4,97,46,1835,20.5,70,europe
peugeot 504,25,4,110,87,2672,17.5,70,europe
audi 100 ls,24,4,107,90,2430,14.5,70,europe
saab 99e,25,4,104,95,2375,17.5,70,europe
bmw 2002,26,4,121,113,2234,12.5,70,europe
amc gremlin,21,6,199,90,2648,15,70,usa
...
```

Appendix II: Algorithm implementations and visualization code

The following code was used to load and process the dataset.

```
import pandas as pd

df = pd.read_csv('cars.csv')
df.head()
```

The following code was used to clean the data and compute the analytical solution using the Levenberg-Marquardt algorithm.

```
from scipy.optimize import curve_fit

# Define the exponential function to fit
def exp(x, a, b, h, k):
    return a * np.e**(b * (x - h)) + k

def loss(y, y_pred):
    return np.sum((y - y_pred) ** 2)

# Initial parameter guess
p0 = [5, -0.05, 90, 12]

# check for non-finite values (NaN, Inf) in the dataset
non_finite_values = df[['horsepower', 'mpg']].isnull().values.any()

# if non-finite values are found, we will clean the dataset
if non_finite_values:
    # remove rows with non-finite values
    df_clean = df.dropna(subset=['horsepower', 'mpg'])
else:
    # if there are no non-finite values, we will proceed with the current dataset
    df_clean = df

# try the curve fit without bounds on the cleaned data
popt, pcov = curve_fit(exp, df_clean['horsepower'], df_clean['mpg'], p0=p0)

# check if the covariance matrix was estimated successfully
if np.any(np.isinf(np.sqrt(np.diag(pcov)))):
```

```

# if the unbounded fit also failed, we will try fitting with bounds and cleaned data.

# define bounds within reasonable ranges based on the data
bounds = ([0, -np.inf, df_clean['horsepower'].min(), 0],
          [np.inf, 0, df_clean['horsepower'].max(), np.inf])

# try the curve fit with bounds on the cleaned data
popt_bound, pcov_bound = curve_fit(exp, df_clean['horsepower'], df_clean['mpg'],
p0=p0, bounds=bounds)

# check if this fit was successful
if np.any(np.isinf(np.sqrt(np.diag(pcov_bound)))):
    # if the bounded fit also failed, we may need to try a different model or
transformation.
    fit_success = False
else:
    fit_success = True
    pop_t = pop_t_bound
    pcov = pcov_bound
else:
    # if the unbounded fit was successful, we will use these parameters.
    fit_success = True

# output the result of the fitting process
fit_success, pop_t, pcov

# create a high-resolution array of x values for a smooth curve
x_high_res = np.linspace(df_clean['horsepower'].min(), df_clean['horsepower'].max(), 1000)
# calculate the fitted y values using the high-resolution x values
y_fitted = exp(x_high_res, *pop_t)

# plot the original data and the fitted curve
plt.figure(figsize=(6, 4), dpi=300)
ax = plt.gca()

# gridlines/ticks
plt.minorticks_on()
ax.tick_params(axis='both', which='major', labelsize=10)
plt.grid(which='both', linestyle='--', linewidth='0.30', color='gray')
plt.grid(which='minor', linestyle=':', linewidth='0.15', color='gray')

# remove top/right spines

```

```

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.scatter(df_clean['horsepower'], df_clean['mpg'], label='Data', marker='x',
            color='grey', s=10)
plt.plot(x_high_res, y_fitted, 'r-', label='Levenberg-Marquardt Fit: a=%5.3f, b=%5.3f,
h=%5.3f, k=%5.3f' % tuple(popt))

plt.xlabel('Horsepower (hp)')
plt.ylabel('mpg (miles/gallon)')
plt.title('Car horsepower vs. fuel efficiency')

plt.legend(fontsize=8, edgecolor='black', facecolor='white', framealpha=0.8,
fancybox=False)

plt.show()

# Actual y values
y_actual = df_clean['mpg']

# Predicted y values from the fitted model
y_pred = exp(df_clean['horsepower'], *popt)

# Calculate the loss
loss_value = loss(y_actual, y_pred)

print(f"The loss for the line of best fit is: {loss_value}")

```

The following code was used to run the two optimization algorithms, keeping track of the loss function and the parameters at each iteration as well as the coefficient of determination R^2 , and generating the visualizations used in the exploration.

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter1d
from tqdm import tqdm

# Exponential model function
def model(x, a, b, h, k):
    return a * np.exp(b * (x - h)) + k

```

```

# Loss function: sum of squared residuals
def loss(y, y_pred):
    return np.sum((y - y_pred) ** 2)

# Optimization function that takes another function as the optimization algorithm
def optimize(x, y, initial_params, num_iterations, optimizer_func, *args, **kwargs):
    param_evolution = []
    loss_evolution = []

    params = initial_params
    T = kwargs.pop('T0', None)
    alpha = kwargs.pop('alpha', None)

    for iteration in tqdm(range(num_iterations)):
        if optimizer_func.__name__ == "simulated_annealing_optimizer":
            params, current_loss, T = optimizer_func(x, y, params, iteration, T, alpha)
        else:
            params, current_loss = optimizer_func(x, y, params, iteration, *args,
**kwargs)

        param_evolution.append(params)
        loss_evolution.append(current_loss)

        if not np.isfinite(current_loss):
            print("Optimization is diverging. Stopping early.")
            break

    return param_evolution, loss_evolution

# Gradient Descent optimizer function
def gradient_descent_optimizer(x, y, params, iteration, learning_rate, max_step):
    a, b, h, k = params
    y_pred = model(x, a, b, h, k)
    current_loss = loss(y, y_pred)

    # Calculate gradients
    grad_a = -2 * np.sum((y - y_pred) * np.exp(b * (x - h)))
    grad_b = -2 * np.sum((y - y_pred) * a * (x - h) * np.exp(b * (x - h)))
    grad_h = 2 * np.sum((y - y_pred) * a * b * np.exp(b * (x - h)))
    grad_k = -2 * np.sum(y - y_pred)

    if iteration == 0:

```



```

    print(grad_a, grad_b, grad_h, grad_k)

# Update parameters
a -= np.clip(learning_rate * grad_a, -max_step, max_step)
b -= np.clip(learning_rate * grad_b, -max_step, max_step)
h -= np.clip(learning_rate * grad_h, -max_step, max_step)
k -= np.clip(learning_rate * grad_k, -max_step, max_step)

return (a, b, h, k), current_loss

# Simulated Annealing optimizer function
def simulated_annealing_optimizer(x, y, params, iteration, T, alpha):
    # result = dual_annealing(objective_function, bounds, maxiter=1, initial_temp=5230,
    restart_temp_ratio=2e-5, visit=2.62, accept=-5.0, no_local_search=True, x0=params)

    # Perturb the current solution
    new_params = params + np.random.normal(0, 1, size=len(params))

    # Calculate loss for the current and new solutions
    current_loss = loss(y, model(x, *params))
    new_loss = loss(y, model(x, *new_params))

    # Calculate the change in loss
    delta_loss = new_loss - current_loss

    # Decide whether to accept the new solution
    if delta_loss < 0 or np.random.rand() < np.exp(-delta_loss / T):
        params = new_params # Accept new solution
        current_loss = new_loss

    # Cooling step (decrease temperature)
    T *= alpha

    return params, current_loss, T

# Parameters for the optimization
initial_params = [50, -0.001, 1, 1]

gd_iterations = 50000
gd_learning_rate = 1e-9
gd_max_step = 1

```

```

sa_iterations = 50000
sa_T0 = 500 # Initial temperature
sa_alpha = 0.98 # Cooling factor

# Run Gradient Descent optimization
print("Running gradient descent...")
gd_param_evolution, gd_loss_evolution = optimize(
    df_clean['horsepower'], df_clean['mpg'], initial_params, gd_iterations,
    gradient_descent_optimizer, learning_rate=gd_learning_rate, max_step=gd_max_step
)

# Run Simulated Annealing optimization
print("Running simulated annealing...")
sa_param_evolution, sa_loss_evolution = optimize(
    df_clean['horsepower'], df_clean['mpg'], initial_params, sa_iterations,
    simulated_annealing_optimizer, T0=sa_T0, alpha=sa_alpha
)

# Function to plot optimization progress
def plot_optimization(x, y, param_evolution, loss_evolution, title, loss_title,
visualize_iterations):
    # Plot for the fitting
    plt.figure(figsize=(6, 4), dpi=300)
    ax = plt.gca()

    # gridlines/ticks
    plt.minorticks_on()
    ax.tick_params(axis='both', which='major', labelsize=10)
    plt.grid(which='both', linestyle='--', linewidth='0.30', color='gray')
    plt.grid(which='minor', linestyle=':', linewidth='0.15', color='gray')

    # remove top/right spines
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    plt.scatter(x, y, label='Data', marker='x', color='grey', s=10)

    # Plot the model fit at selected iterations
    for i in visualize_iterations:
        # Get the parameters from the ith iteration
        a, b, h, k = param_evolution[i]

```

```

# Generate the model predictions
x_values = np.linspace(x.min(), x.max(), 500)
y_values = model(x_values, a, b, h, k)

# Plot
plt.plot(x_values, y_values, label=f'Iteration {i+1}', linewidth=0.5)

plt.xlabel('Horsepower (hp)')
plt.ylabel('mpg (miles/gallon)')
plt.title(title)
plt.legend(fontsize=8, fancybox=False)
plt.show()

# Plot the evolution of the loss with smoothing
smoothed_loss = gaussian_filter1d(loss_evolution, sigma=50)

plt.figure(figsize=(6, 4), dpi=300)
plt.plot(smoothed_loss, linewidth=0.5)
plt.ylim(0, 80000)
plt.title(loss_title)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.grid(True)

ax = plt.gca()

# gridlines/ticks
plt.minorticks_on()
ax.tick_params(axis='both', which='major', labelsize=10)
plt.grid(which='both', linestyle='--', linewidth='0.30', color='gray')
plt.grid(which='minor', linestyle=':', linewidth='0.15', color='gray')

# remove top/right spines
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.show()

# Function to plot the evolution of loss for both optimization algorithms
def plot_loss_evolution(gd_loss_evolution, sa_loss_evolution, title):
    # Plot Gradient Descent loss evolution
    plt.figure(figsize=(6, 4), dpi=300)

```

```

smoothed_loss_gd = gaussian_filter1d(gd_loss_evolution, sigma=50)
plt.plot(smoothed_loss_gd, label='Gradient Descent', linewidth=0.5)

# Plot Simulated Annealing loss evolution
smoothed_loss_sa = gaussian_filter1d(sa_loss_evolution, sigma=50)
plt.plot(smoothed_loss_sa, label='Simulated Annealing', linewidth=0.5)

plt.ylim(0, 80000)
plt.title(title)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.legend(fontsize=8, fancybox=False)
plt.grid(True)
plt.show()

# Plot the results for Gradient Descent
gd_selected_iterations = [0] + list(range(9999, gd_iterations+1, 10000))
print(gd_selected_iterations)
plot_optimization(df_clean['horsepower'], df_clean['mpg'], gd_param_evolution,
gd_loss_evolution, 'Gradient Descent Optimization of Model Fit', 'Evolution of Loss for
Gradient Descent', gd_selected_iterations)

# Plot the results for Simulated Annealing
sa_selected_iterations = [0] + list(range(9999, sa_iterations+1, 10000))
plot_optimization(df_clean['horsepower'], df_clean['mpg'], sa_param_evolution,
sa_loss_evolution, 'Simulated Annealing Optimization of Model Fit', 'Evolution of Loss for
Simulated Annealing', sa_selected_iterations)

# compare loss between both
plot_loss_evolution(gd_loss_evolution, sa_loss_evolution, 'Comparison: Evolution of Loss')

# Plot of the two fits on the same scatter plot
plt.figure(figsize=(6, 4), dpi=300)
ax = plt.gca()

# gridlines/ticks
plt.minorticks_on()
ax.tick_params(axis='both', which='major', labelsize=10)
plt.grid(which='both', linestyle='--', linewidth='0.30', color='gray')
plt.grid(which='minor', linestyle=':', linewidth='0.15', color='gray')

# remove top/right spines

```

```

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.scatter(df_clean['horsepower'], df_clean['mpg'], label='Data', marker='x',
            color='grey', s=10)

from sklearn.metrics import r2_score

# Gradient Descent final fit
gd_final_params = gd_param_evolution[-1]
x_values = np.linspace(df_clean['horsepower'].min(), df_clean['horsepower'].max(), 500)
gd_y_values = model(x_values, *gd_final_params)
plt.plot(x_values, gd_y_values, label='Gradient Descent Fit', linewidth=0.5)

# Simulated Annealing final fit
sa_final_params = sa_param_evolution[-1]
sa_y_values = model(x_values, *sa_final_params)
plt.plot(x_values, sa_y_values, label='Simulated Annealing Fit', linewidth=0.5)

# theoretical fit
lm_final_params = [41.927, -0.016, 24.890, 9.453]
lm_y_values = model(x_values, *lm_final_params)
plt.plot(x_values, lm_y_values, label='Levenberg-Marquardt Fit', linewidth=0.5)

# Calculate and print R^2 value for the Levenberg-Marquardt fit
lm_r2 = r2_score(df_clean['mpg'], model(df_clean['horsepower'], *lm_final_params))
print(f"R^2 value for Levenberg-Marquardt Fit: {lm_r2}")

plt.xlabel('Horsepower (hp)')
plt.ylabel('mpg (miles/gallon)')
plt.title('Comparison of Model Fits')
plt.legend(fontsize=8, fancybox=False)
plt.show()

def report(gd_report_iterations, sa_report_iterations):
    gd_report_data = {
        'Iteration': gd_report_iterations,
        'Algorithm': 'GD',
        'a': [gd_param_evolution[i][0] if i < len(gd_param_evolution) else None for i in
gd_report_iterations],
        'b': [gd_param_evolution[i][1] if i < len(gd_param_evolution) else None for i in
gd_report_iterations],

```

```

    'h': [gd_param_evolution[i][2] if i < len(gd_param_evolution) else None for i in
gd_report_iterations],
    'k': [gd_param_evolution[i][3] if i < len(gd_param_evolution) else None for i in
gd_report_iterations],
    'Loss': [gd_loss_evolution[i] if i < len(gd_loss_evolution) else None for i in
gd_report_iterations],
    'R2': [r2_score(df_clean['mpg'], model(df_clean['horsepower'],
*gd_param_evolution[i])) if i < len(gd_param_evolution) else None for i in
gd_report_iterations]
}

sa_report_data = {
    'Iteration': sa_report_iterations,
    'Algorithm': 'SA',
    'a': [sa_param_evolution[i][0] if i < len(sa_param_evolution) else None for i in
sa_report_iterations],
    'b': [sa_param_evolution[i][1] if i < len(sa_param_evolution) else None for i in
sa_report_iterations],
    'h': [sa_param_evolution[i][2] if i < len(sa_param_evolution) else None for i in
sa_report_iterations],
    'k': [sa_param_evolution[i][3] if i < len(sa_param_evolution) else None for i in
sa_report_iterations],
    'Loss': [sa_loss_evolution[i] if i < len(sa_loss_evolution) else None for i in
sa_report_iterations],
    'R2': [r2_score(df_clean['mpg'], model(df_clean['horsepower'],
*sa_param_evolution[i])) if i < len(sa_param_evolution) else None for i in
sa_report_iterations]
}

# Convert the dictionaries to DataFrames
gd_report_df = pd.DataFrame(gd_report_data)
sa_report_df = pd.DataFrame(sa_report_data)

# Concatenate the Gradient Descent and Simulated Annealing report DataFrames
report_df = pd.concat([gd_report_df, sa_report_df])

# Sort by 'Iteration' and reset the index
report_df.sort_values(by='Iteration', inplace=True)
report_df.reset_index(drop=True, inplace=True)

# Display the combined DataFrame
report_df.sort_values(by='Iteration', inplace=True)

```

```
report_df.reset_index(drop=True, inplace=True)  
report_df
```

```
return report_df
```

```
report_df = report(gd_selected_iterations, sa_selected_iterations)  
report_df
```